

Programación Orientada a Objetos

Ingeniería Civil en Informática

Funciones

Definición y explicación gráfica de su ejecución

21 de Agosto del 2026



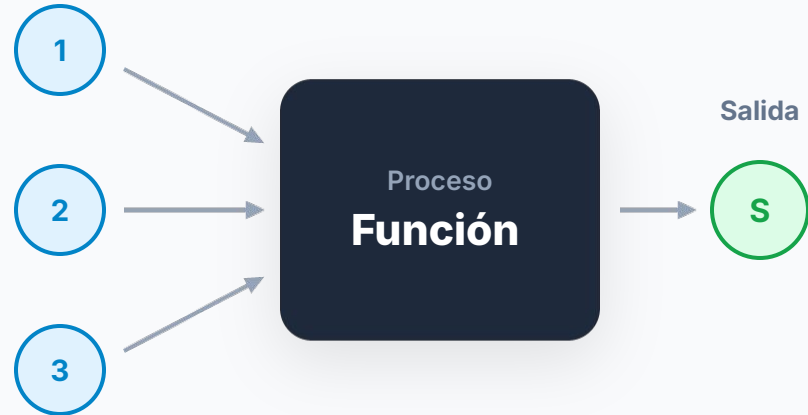
Programación Orientada a Objetos

Ingeniería Civil en Informática

¿Qué es una Función?

De manera informal, una función es una **caja** que recibe **"cosas" (entradas)** y produce una salida (**otra "cosa"**) empleando esos elementos de entrada.

Entradas



Programación Orientada a Objetos

Ingeniería Civil en Informática

¿Cómo funciona?

Si las cosas fueran **bloques de construcción**, una función común y altamente estructurada podría ser la de **apilar**.



Es importante distinguir entre:



Funciones que ya existen

Están integradas en las librerías del lenguaje (como [stdio.h](#)). Solo es necesario invocarlas o llamarlas cuando se requieren.



Creadas por el programador

Definidas a medida por el desarrollador para resolver problemas específicos, estructurando su propia cabecera y comportamiento.



Funciones que ya existen



Invocación Directa

Están integradas en librerías y listas para usar. Solo tienes que llamarlas por su nombre en tu código, sin necesidad de programar su lógica interna.



La Librería stdio.h

Es la biblioteca estándar de entrada y salida en C. Contiene funciones predefinidas como **printf**, que se utiliza para imprimir texto en la consola de comandos.



Las creadas por el programador

Tres pasos siempre: llamada, cabecera, ejecución

```
#include <stdio.h>
```

```
int sumar(int a, int b)
```

```
{  
    return a + b;  
}
```

```
int main()  
{
```

```
    int resultado;
```

```
    resultado = sumar(5, 3);
```

```
    printf("Resultado: %d\n", resultado);
```

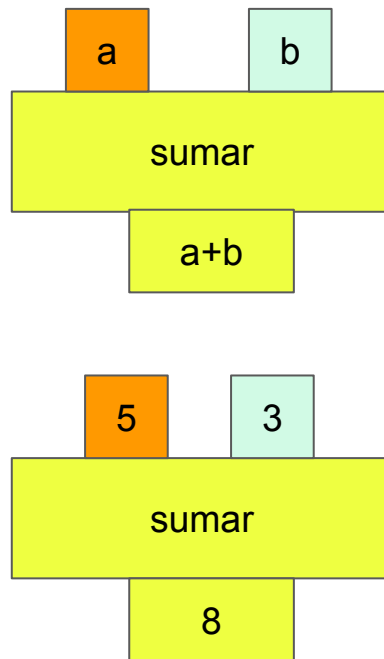
```
    return 0;
```

```
}
```

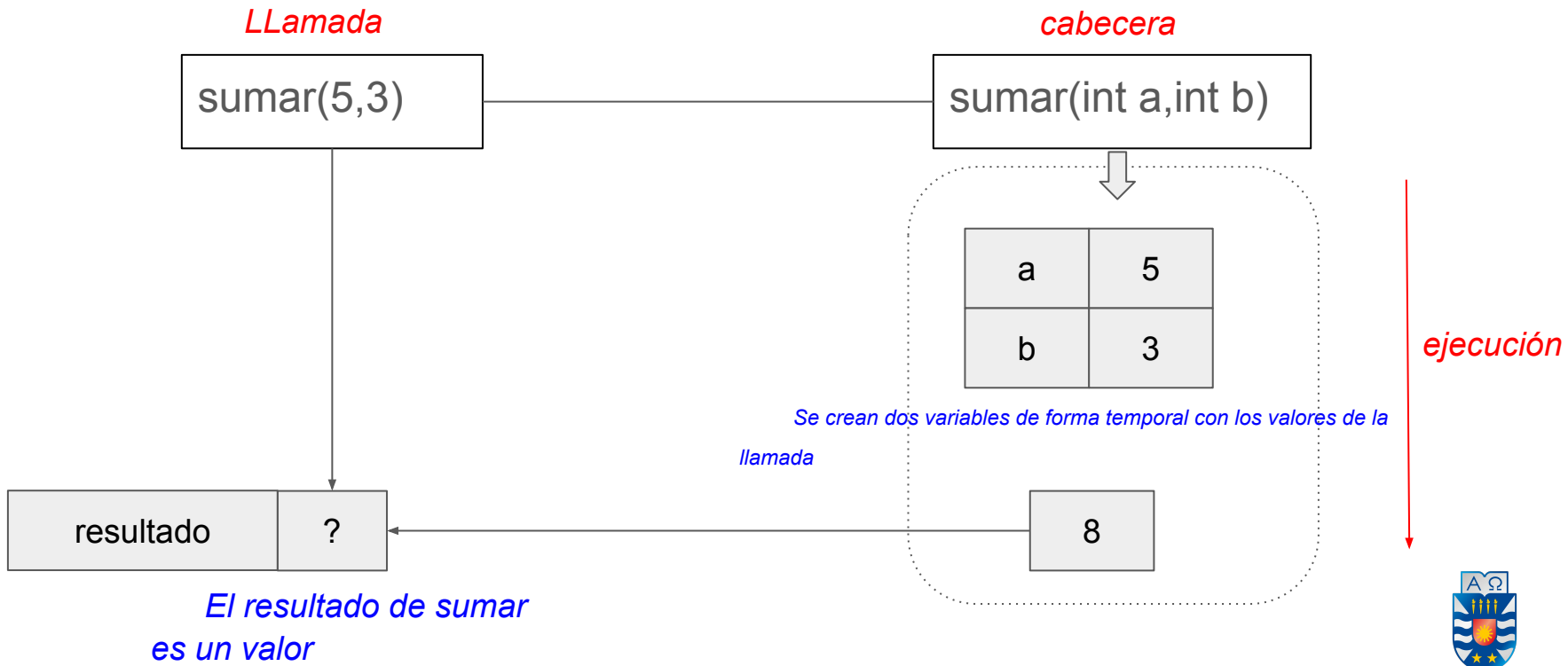
La cabecera

Ejecución

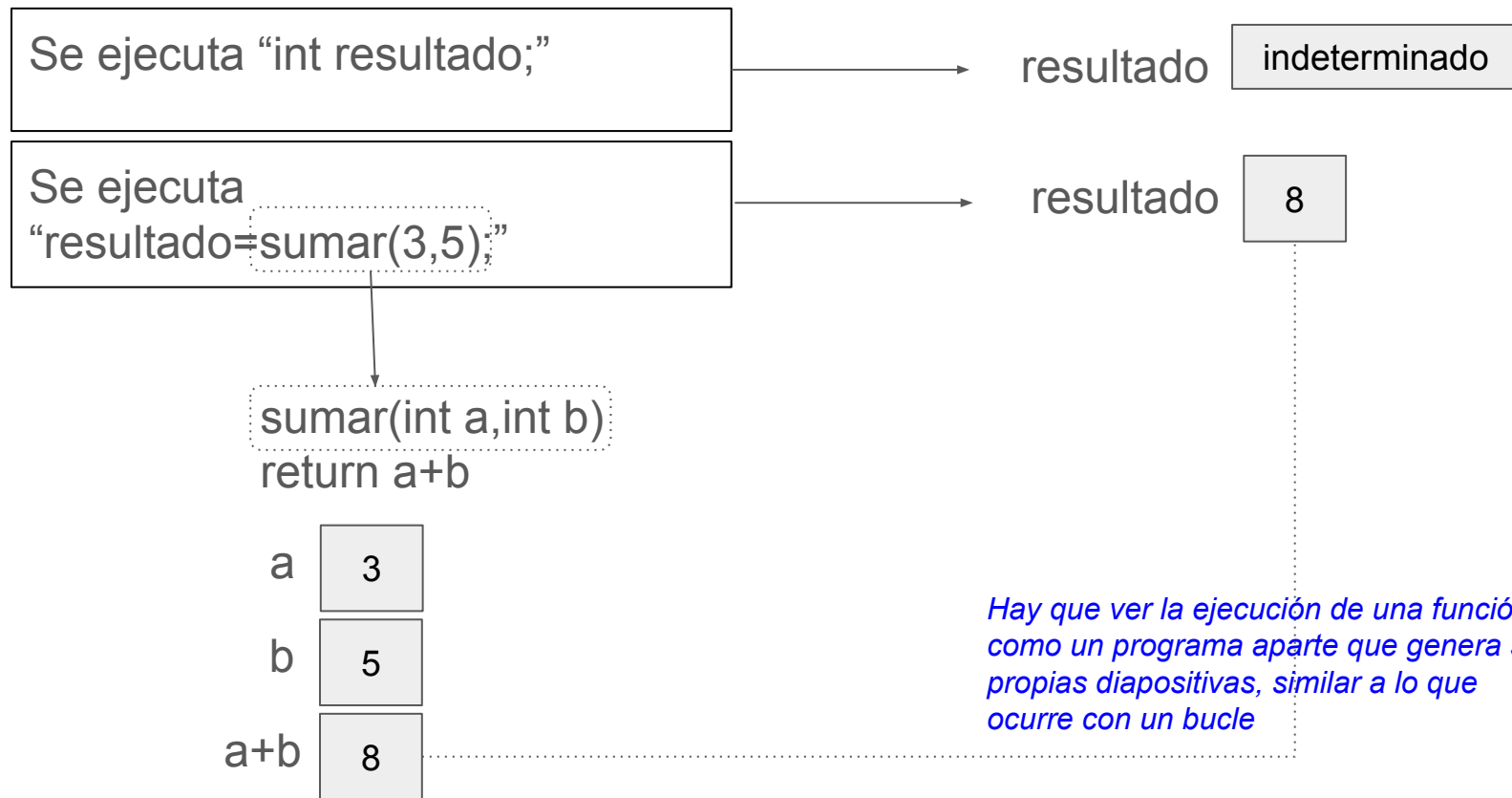
La llamada



Explicación: emparejamiento llamada - cabecera - ejecución



En una explicación de la ejecución



Funciones y arrays



```
1  #include <stdio.h>
2
3  void modificar(int a[],int pos, int valor) {
4
5      a[pos]=valor;
6  }
7
8  int main(void) {
9
10     int array[]={ 1, 2, 3 };
11     int n = sizeof(array)/sizeof(array[0]);
12
13     for (int i = 0; i < n; i++) {
14         printf(format: "Elemento %d: %d\n", i, array[i]);
15     }
16
17     modificar(array, pos: 0, valor: 0);
18
19     for (int i = 0; i < n; i++) {
20         printf(format: "Elemento %d: %d\n", i, array[i]);
21     }
22     return 0;
23 }
```

La cabecera

a

pos

valor

modificar

void

//no devuelve nada modifica

La llamada

array

0

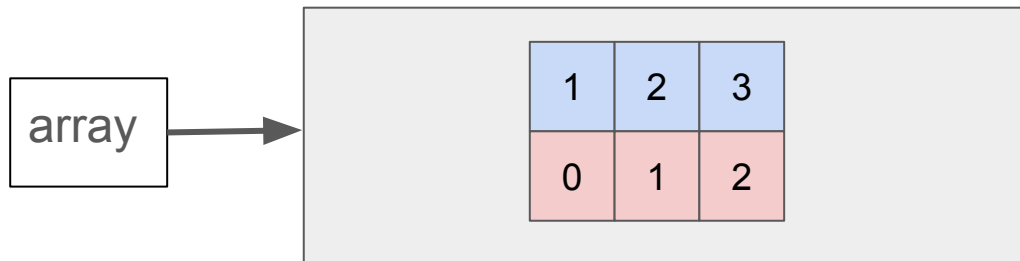
0

modificar

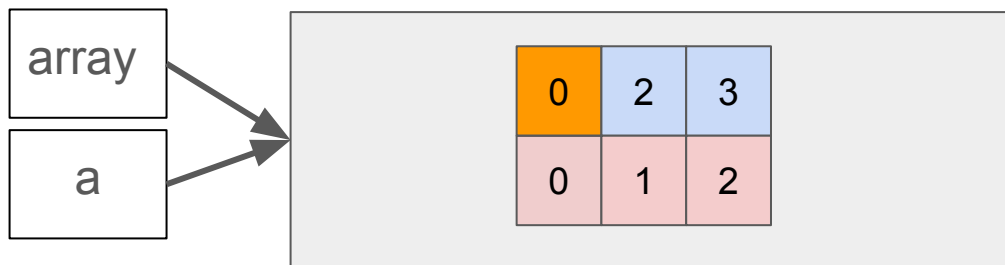
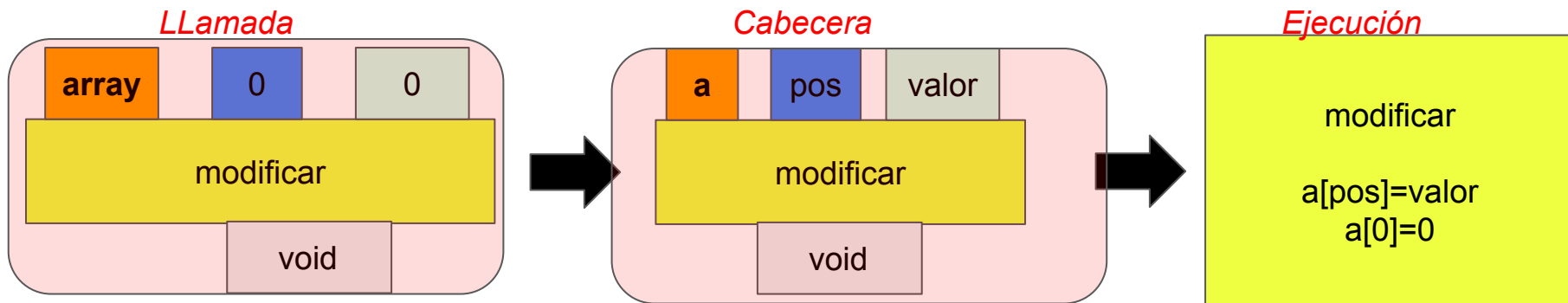
void

//no devuelve nada modifica

Qué hace modificar



Usaremos **una flecha** para hacer referencia al hecho de que cuando pasamos array a un función pasamos la zona donde se ha guardado la posición y los valores . Es decir, un array se pasa por referencia y no por valor



pos	0
valor	0

Tarea 3

Implementación de una función que permita **concatenar** dos arrays cualquiera.

Explicar su ejecución paso a paso. Suponiendo que:

```
int array1[]={ 0 , 1 , 0 };
```

```
int array2[]={ 1, 0 , 1 };
```

```
concatenar(array1,array2)=[0,1,0,1,0,1]
```

```
union(array1,array2)=[0,1]
```



Detalle tener en cuenta

Uno podría pensar que la función sería algo así:

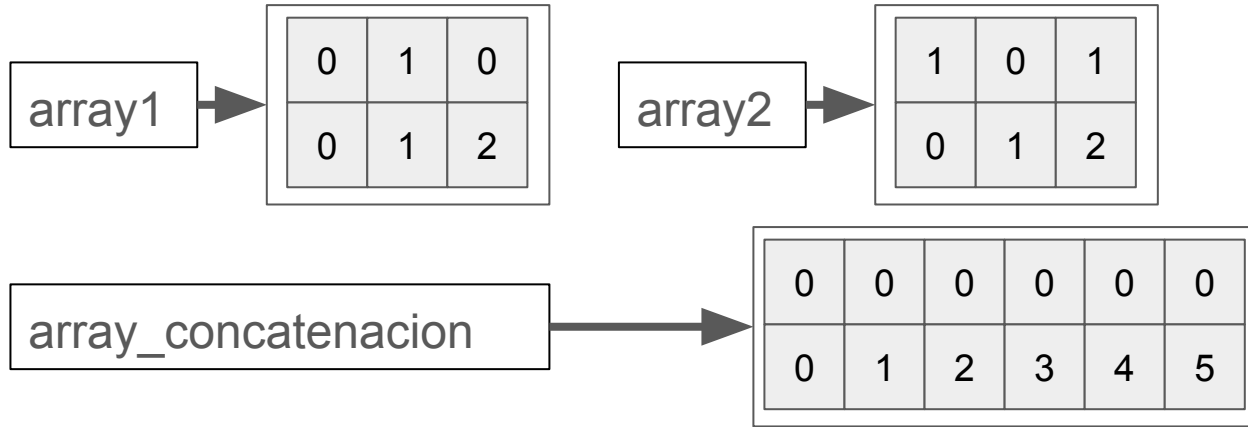
```
array concatenar(array1, array2) //función que devuelve un array
```

Una función en C no puede devolver de forma segura un array local, porque deja de existir cuando la función termina. Sin embargo, puede devolver un puntero a memoria dinámica o a un array estático.

La opción más segura parece ser que pasar como argumento el array donde quiero que se produzca la unión.

```
1  #include <stdio.h>
2
3  void concatenar(int array1[], int n1,
4                  int array2[], int n2,
5                  int resultado[]) {
6
7      for (int i = 0; i < n1; i++) {
8          resultado[i] = array1[i];
9      }
10
11     for (int i = n1, j=0; i < n1+n2; i++,j++) {
12         resultado[i] = array2[j];
13     }
14 }
15
16 int main(void) {
17
18     int array1[] = {1, 2, 3};
19     int array2[] = {4, 5, 6};
20
21     int array_concatenacion[6]={};
22
23     concatenar(array1, n1:3, array2, n2:3, resultado: array_concatenacion);
24
25     for (int i = 0; i < 6; i++) {
26         printf(format: "Elemento %d: %d\n", i, array_concatenacion[i]);
27     }
28
29     return 0;
30 }
```

Instrucción 8, 9, 11

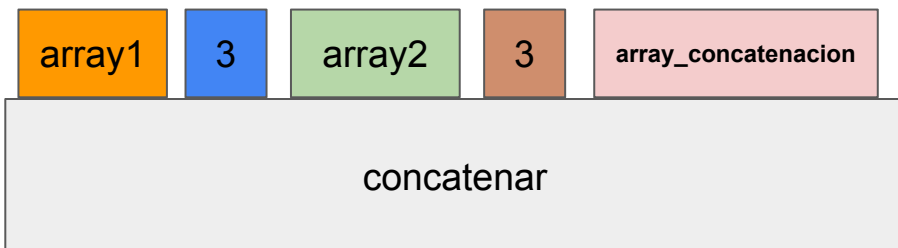


Instrucción 21:

```
concatenar(array1, 3, array2, 3, array_concatenacion);
```

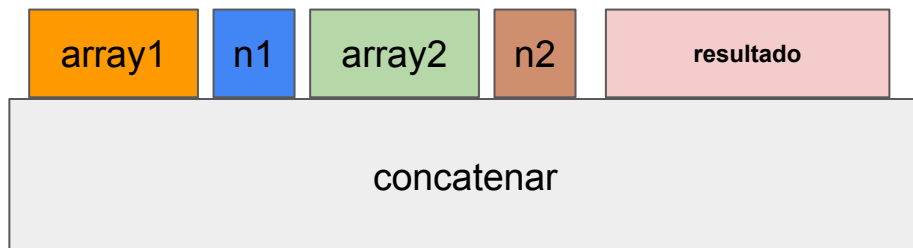
LLamada

concatenar(array1,3,array2,3,array_concatenacion)

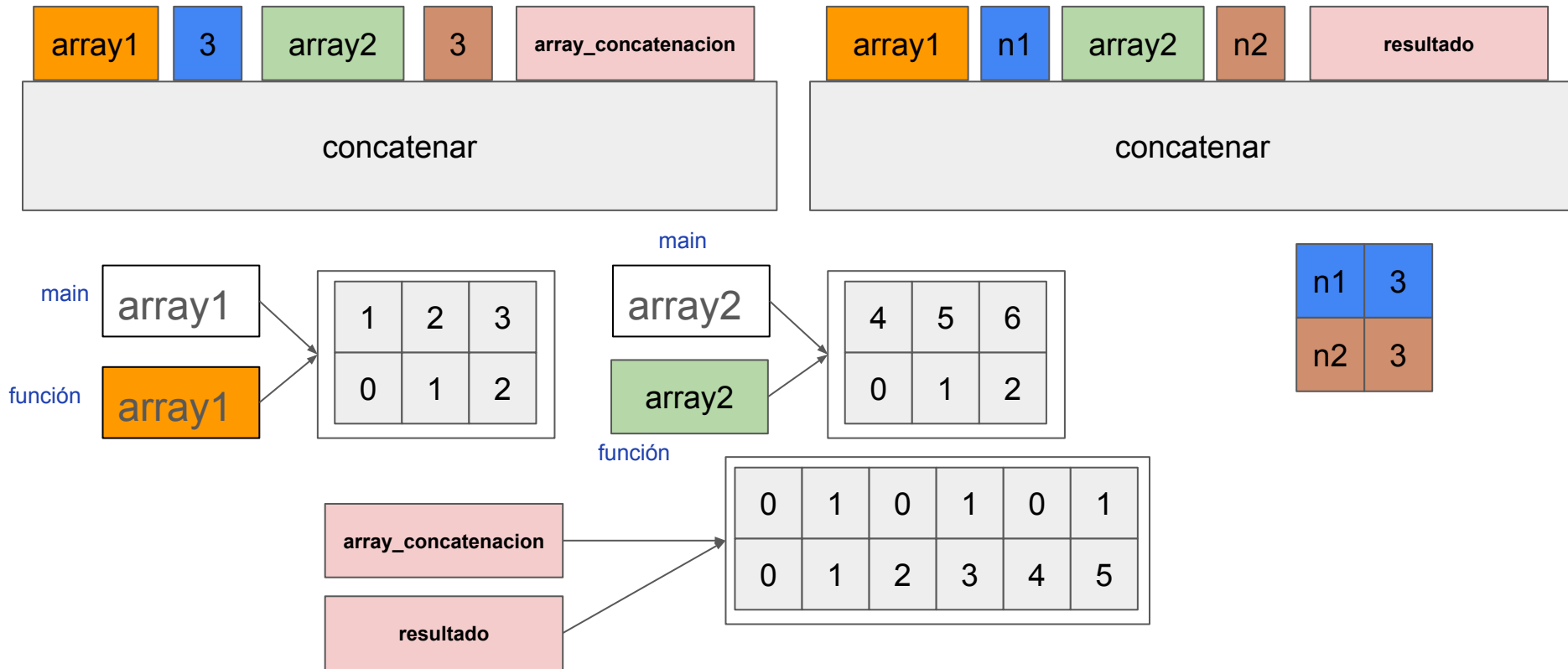


Cabecera

```
void concatenar(int array1[], int n1,  
int array2[], int n2,  
int resultado[])
```



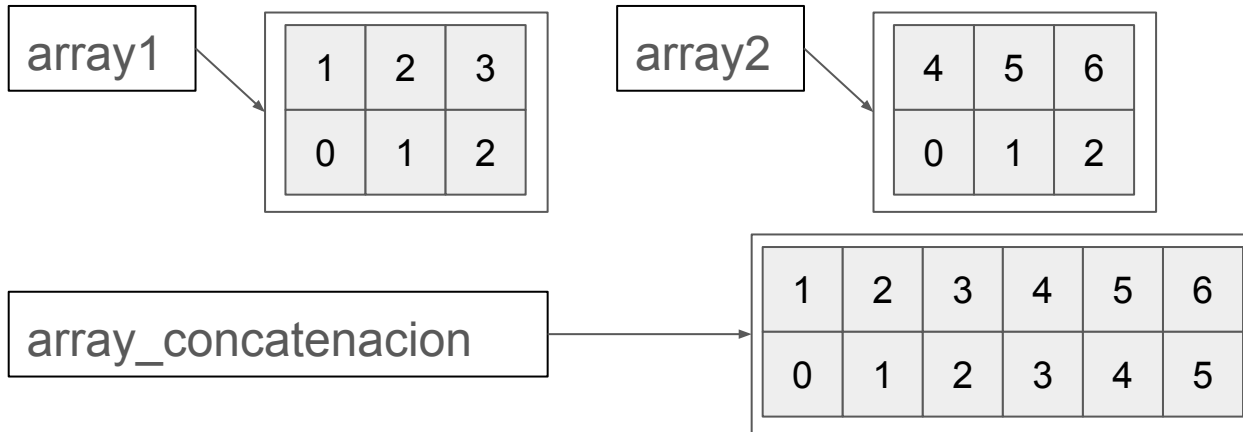
Instrucción 21: Simulación del emparejamiento



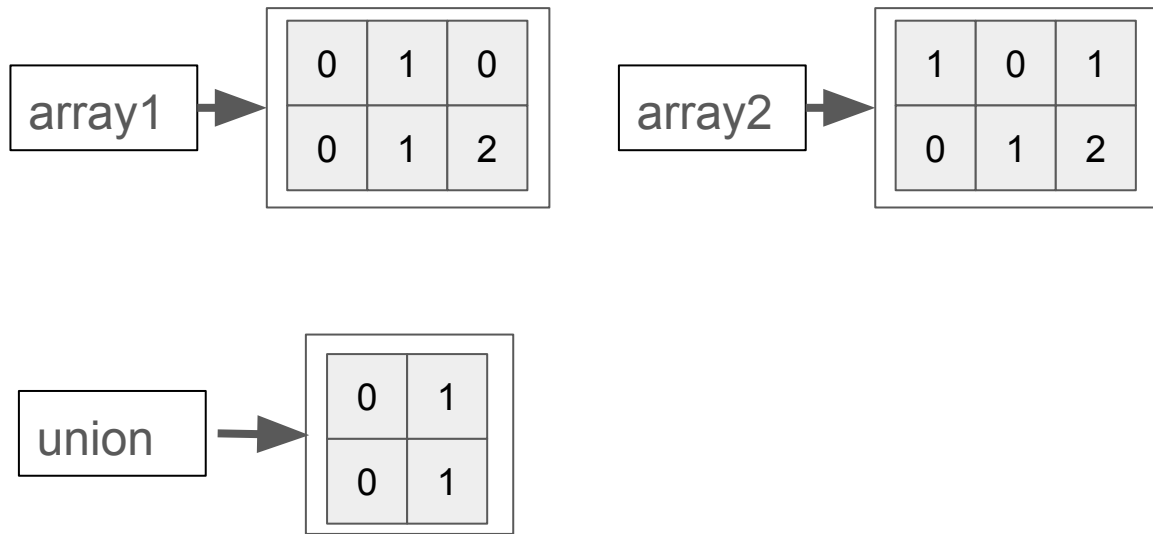
Ejecución de la función (sería como la tarea 2)

“los parámetros de una función son **variables locales de la función**, cuya existencia está asociada a la ejecución de esa función.”

aquí pongo el resultado



Cómo sería la unión



```
3 int unir(int array1[], int n1, int array2[], int n2, int array_union[]) {
4     int i, j;
5     int encontrado;
6
7     // Copiar elementos de array1 sin repetir
8     for (i = 0; i < n1; i++) {
9         encontrado = 0;
10
11         for (j = 0; j < n_union; j++) {
12             if (array_union[j] == array1[i]) {
13                 encontrado = 1;
14             }
15         }
16
17         if (!encontrado) {
18             array_union[n_union] = array1[i];
19             n_union++;
20         }
21     }
22
23     // Añadir elementos de array2 que no estén en union
24     for (i = 0; i < n2; i++) {
25         encontrado = 0;
26
27         for (j = 0; j < n_union; j++) {
28             if (array_union[j] == array2[i]) {
29                 encontrado = 1;
30             }
31         }
32
33         if (!encontrado) {
34             array_union[n_union] = array2[i];
35             n_union++;
36         }
37     }
38
39     return n_union;
40 }
41 }
```

```
43 int main(void) {
```

```
45     int array2[] = {0, 1, 0};
```

```
46     int array3[] = {1, 0, 1};
```

```
47     int array_union[6];
```

```
49     int n_union = unir(array1: array2, n1: 3, array2: array3, n2: 3, array_union);
```

```
51     for (int i = 0; i < n_union; i++) {
```

```
52         printf(format: "Elemento %d: %d\n", i, array_union[i]);
```

```
55     return 0;
```

```
57 }
```